Compte rendu : TP Android – ListView de contacts modifiable

1. Introduction

Ce TP a pour objectif de réaliser une application Android en Kotlin permettant d'afficher et de gérer une liste de contacts à l'aide du composant ListView. L'utilisateur peut sélectionner un contact, modifier son nom, ajouter de nouveaux contacts et supprimer ceux dont il n'a plus besoin.

2. Outils et composants utilisés

- Android Studio : environnement de développement de l'application.
- Kotlin : langage utilisé pour programmer le comportement de l'application.
- XML : langage utilisé pour construire l'interface.
- ListView : composant qui affiche les contacts sous forme de liste.
- ArrayAdapter : adaptateur qui relie la liste de données à la ListView.
- EditText et Button : composants utilisés pour saisir un nom et déclencher une action.
- Toast et AlertDialog : composants utilisés pour afficher des messages et demander une confirmation.
- ViewModel : composant utilisé pour conserver les données lors d'un changement de configuration, comme la rotation de l'écran.

3. Réalisation de l'interface

L'interface est construite avec un LinearLayout vertical. Elle contient un champ EditText pour saisir ou afficher le nom d'un contact, deux boutons intitulés « Ajouter » et « Modifier », ainsi qu'une ListView qui occupe l'espace restant de l'écran. Les éléments de l'interface sont identifiés par des identifiants afin de pouvoir être récupérés dans le code Kotlin.

4. Fonctionnement de l'application

Les contacts sont stockés dans une liste Kotlin modifiable créée avec mutableListOf(). Un ArrayAdapter est ensuite associé à la ListView pour afficher les noms. Lorsqu'un utilisateur clique sur un élément, sa position est mémorisée et son nom est copié dans le champ EditText.

Pour modifier un contact, l'application vérifie qu'un élément a été sélectionné et que le nouveau nom n'est pas vide. Elle vérifie également que ce nom n'existe pas déjà dans la liste. Après la modification, notifyDataSetChanged() actualise l'affichage.

5. Exercices complémentaires

- Ajout : le bouton « Ajouter » insère le nom saisi dans la liste après vérification qu'il n'est pas vide et qu'il n'existe pas déjà.
- Suppression : un appui long sur un contact ouvre une boîte de dialogue AlertDialog. La suppression est effectuée uniquement après confirmation.
- Gestion des doublons : les comparaisons de noms ignorent les différences entre majuscules et minuscules.
- Tri : les contacts sont classés par ordre alphabétique après un ajout ou une modification.
- Rotation de l'écran : un ViewModel conserve la liste pendant la recréation de l'activité liée à un changement de configuration.

6. Tests effectués

Les tests consistent à afficher les contacts initiaux, sélectionner un contact, modifier son nom, ajouter un nouveau contact, tenter d'ajouter un doublon, supprimer un contact avec confirmation et vérifier le classement alphabétique. La rotation de l'écran permet également de vérifier que les données restent disponibles pendant la recréation de l'activité.

7. Conclusion

Ce TP a permis de mettre en pratique l'utilisation de ListView et ArrayAdapter dans une application Android développée en Kotlin. Il a également permis de manipuler une liste modifiable, de gérer les événements de clic, de contrôler les saisies et d'actualiser l'interface après chaque opération. Les exercices complémentaires ont ajouté des fonctionnalités de gestion des contacts et une première utilisation de ViewModel.